Föreläsning 12

Giriga algoritmer

Föreläsning 12

- Giriga algoritmer
- Användning
- Växelproblemet
- Kappsäcksproblemet
- Schemaläggning
- Färgläggning
- Handelsresandeproblemet

Giriga algoritmer (Greedy algorithms)

En girig algoritm börjar med att lösa ett delproblem. Den löser detta delproblem genom att välja den lokalt bästa lösningen utan att ta hänsyn till det globala problemet. Den arbetar sedan vidare i steg med delproblem så att den till slut har löst hela problemet. För att verkligen klassificeras som en girig algoritm bör den inte heller omvärdera de lokala lösningar den gjort (Dijkstra anses ibland girig men den omvärderar gjorda val).

Ex. från förra föreläsningen: Börja med en stad, anslut närmsta stad, anslut närmsta stad till de två städerna, …

Användningsområden

Beroende på problem ger en girig algoritm den optimala lösningen, en hyggligt bra lösning inom rimlig tid eller en helt usel lösning.

När en girig algoritm fungerar är den ofta ett effektivt alternativ (jämfört t.ex med dynamisk programmering). Det finns problem där man inte får den optimala lösningen men där det tar för lång tid att hitta denna optimala lösningen (traveling salesman problem).

Svåra och vanliga problem är normalt redan lösta och en ingenjör behöver kunna implementera, anpassa eller bara hitta implementeringar av redan skapade och välprövade algoritmer

Däremot stöter man då och då på relativt enkla problem som kräver en lösning – oftast behöver man inte hitta den bästa lösningen utan det räcker att man hittar en lösning

Det är bl.a. för dessa problem man slipar sin problemlösningslåda – giriga algoritmer är ett exempel på en sorts lösningar som ofta är effektiva nog i dessa situationer och som är ganska enkla att konstruera.

Växelproblemet

Vad är det minsta antalet mynt och sedlar som krävs för att ge växel för ett givet belopp?

Fundera ut hur en girig algoritm skulle se ut.

En girig algoritm för att ge växel

Antag att vi vill ge x kr i växel

n = 0

Medans x > 0

Sätt p = den största valören $\leq$ x
Sätt x = x – p
Sätt n = n + 1

Svar: n

Fungerar den? Ger den bästa resultat?

Ex: 77 kr: 50, 20, 5, 1, 1: n = 5

Varför är algoritmen girig?

Ex: Ge växel på 30 kr med valörerna 1, 10, 25

Vi återkommer till detta när det är dags för dynamisk programmering.

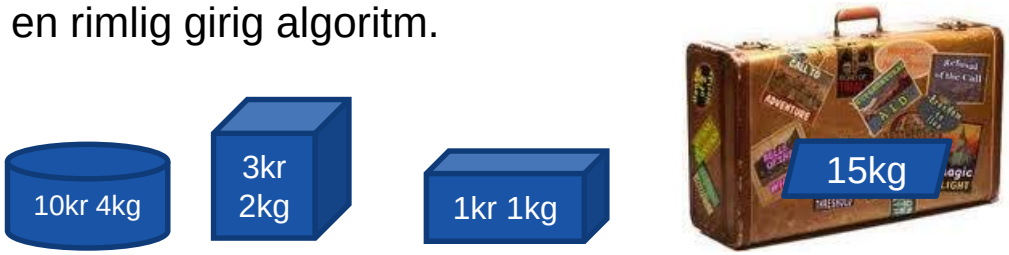
Kappsäcksproblemet (knapsack)

Det finns ett visst antal olika sorters varor *n* vars värde är v_i och vikt är w_i. Hur ska vi packa en kappsäck som maximalt tar vikten W så att vi får med oss det maximala värdet?

Det här gamla problemet med moderna tillämpningar såsom (schemaläggning, flygplansrutter och produktionsplanering) finns i flera olika versioner. Vi ska här titta på den version som kallas obegränsad: Av varje sorts vara finns det obegränsat antal.

För denna version är en girig algoritm garanterad att få åtminstone halva det maximala värdet (om det endast finns en begränsad mängd av varje vara kan den lyckas mycket sämre). Även för detta problem kan man hitta den optimala lösningen med dynamisk programmering.

Formulera en rimlig girig algoritm.



Girig algoritm för kappsäcksproblemet

Fyll kappsäcken med så många som möjligt av den vara som har högst värde per viktenhet (kr/kg).

Fyll kappsäcken med så många som möjligt av den vara som har näst högst värde per viktenhet.

osv

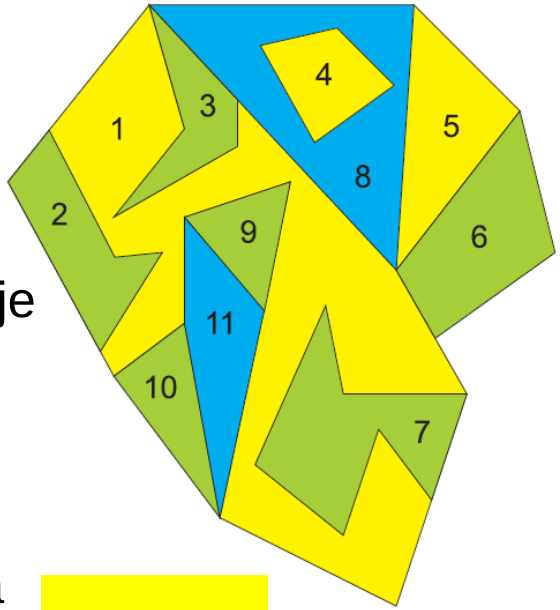
Försök konstruera ett exempel där algoritmen misslyckas.

Ex: kappsäck 20kg

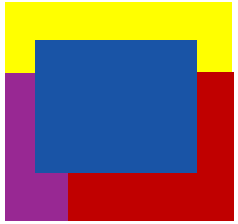
12 kr 11kg
10 kr 10kg.

Färgläggning av karta

(vertex coloring)



- Om man vill färglägga en karta med länder så att varje landområde inte angränsar till något område med samma färg klarar man sig oftast med tre färger.



- Matematiskt har man (med stor möda) lyckats bevisa att det aldrig behövs mer än fyra färger (förutsatt att varje land sitter ihop).

- Det finns ingen algoritm som garanterat hittar den bästa lösningen. En girig algoritm kommer på kort tid ofta hitta en ganska bra lösning.

- Kom ihåg: länderna representeras av noder och att de angränsar av en båge i grafteori

Tillämpningar

Noderna (länderna) representerar kurser och bågarna (gränser) representerar att två klasser har gemensamma studenter. Olika färger motsvarar då att två klasser inte kan schemaläggas samtidigt. (Detta problem kan ge grafer som inte motsvaras av en karta och där lösningen kan behöva fler än 4 färger.)

Noderna representerar radiostationer, och bågarna att de är nära nog att interferera med varandra och olika färg att de behöver använda olika frekvensband.

Noderna representerar trafikljus i en korsning och bågarna att två ljus inte kan vara gröna samtidigt.

En girig algoritm för färgläggning av karta

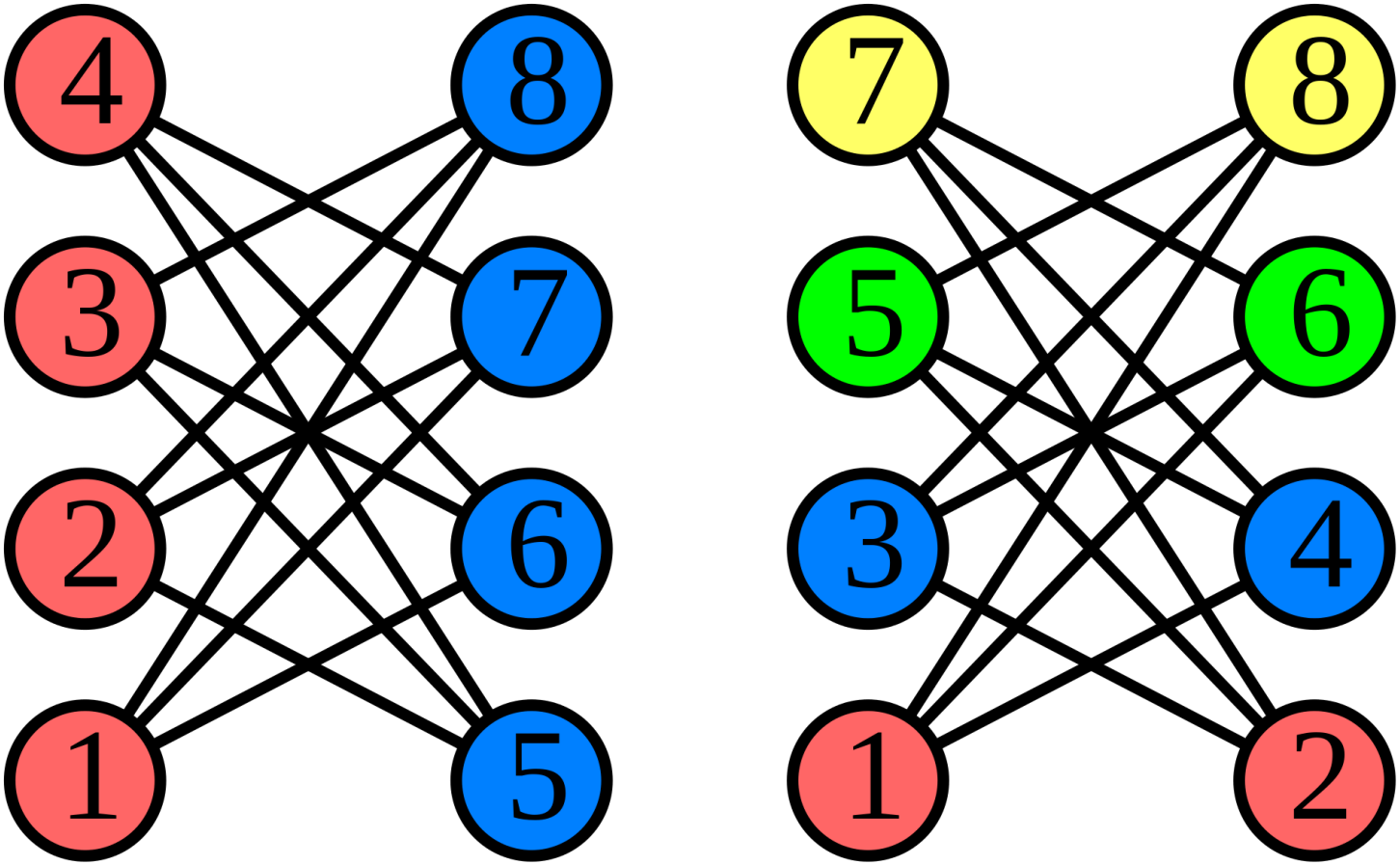
Ordna färgerna (kalla dem 1,2,3,4,…)

Så länge det ännu finns icke färglagda länder

- Välj ett land
- Ge landet den första färgen som inget angränsande land har

Hur bra denna presterar beror på i vilken ordning man väljer länderna. Man kan visa att det för en given graf finns en ordning där antalet färger blir optimalt. Men det finns också exempel där vi får sämsta möjliga resultat. Olika strategier för hur vi väljer ordning ger upphov till olika giriga algoritmer.

Bästa kontra sämsta ordningen



By Thore Husfeldt - Own work, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=5399420

Implementering

För en någorlunda enkel implementering

Indata:

Låt oss kalla länderna 0, 1, 2, …, n-1 och låt gränser[i][j] vara true om det finns en gräns mellan land i och land j och annars false.

Utdata:

Kalla färgerna 1, 2, 3,…, m. Låt land[i] vara den färg land i har fått.

Intern data:

färg[i] är true om ett visst land angränsar till ett land med färg i

Algoritm:

```
Sätt land[i] = 0 för alla i
För i = 0 till n-1//går igenom länderna så att vi hanterar land i nedan:
        Sätt färg[j]= false för alla j
        För j = 0 till n-1//går igenom länderna för att hitta grannar till land i
                om gränser[i][j] och land[j]!=0
                        Sätt färger[land[j]]=true //markerar angränsande färger
        För j = 1 till m //går igenom färgerna för att hitta första lediga
                om inte färger[j]
                        land[i] = j
                        avbryt for-loopen
```

Ett schemaläggningsproblem

Vi har en sal och ett antal aktiviteter under en dag. Varje aktivitet har en starttid och en sluttid. Målet är att schemalägga så många aktiviteter som möjligt.

En girig algoritm:

V – mängden av alla aktiviteter

Medans det finns aktiviteter i V

Schemalägg den aktivitet i V som slutar först.

Tag bort vald aktivitet ur V

Tag bort alla aktiviteter ur V som överlappar med vald aktivitet

Är lösningen den optimala lösningen? Om inte kan du hitta ett motexempel?

Handelsresandeproblemet

Givet ett antal städer och avståndet mellan varje par av städer. Vilken väg ska en handelsresande välja för att minimera resvägen och besöka alla städer och börja och sluta i samma stad.

Problemet berör inte bara ren logistik utan uppkommer även t.ex inom nätverksanalyser.

Att hitta den optimala lösningen är ett mycket krävande problem för stora system. En girig algoritm är inte garanterad att hitta den bästa lösningen men går väldigt fort.

Formulera en girig algoritm.

En girig algoritm för handelsresandeproblemet (nearest neighbour algorithm)

Välj en startstad

Medans det finns obesökta städer

    Res till den stad som ligger närmast den du befinner dig i

Res till startstaden

Problem: Den lägger inte upp en rutt och de sista resorna blir lätt väldigt långa. Den passar inte på att ta alla städer där den är utan åker vidare om det ger tillfälligt kortare resa. Dessutom ser den inte till att avsluta bredvid startstaden. I genomsnitt blir rutten 25% längre än den optimala men det går att konstruera fall där den ger sämsta möjliga väg.

Huffmankodning

Det vanligaste sättet att koda tecken är att ha ett bestämt antal bitar för att representera varje tecken, tex ASCII.

Huffmankodning minimerar filstorleken genom att se till att ju oftare ett tecken förekommer ju kortare kod får det.

Huffman-algoritmen för att tilldela koder kan anses vara girig.

Läs gärna mer om denna komprimeringsteknik i boken.